

Compressed Sensing with Deep Image Prior for Chest CT Signal Denoising

Advanced Techniques & Future Work Guide

Months 2–4: Architecture Design, Regularization & Hybrid Models

Project Team

February 2026

Abstract

This document serves as a technical guide for the remaining three months of the Deep Image Prior (DIP) project. Building on the classical baselines established in Month 1, we outline a comprehensive set of advanced techniques spanning architectural innovations, loss function design, regularization strategies, and hybrid generative models. Each proposition is presented with its mathematical formulation, implementation strategy, expected strengths and weaknesses, and recommended hyperparameters. The guide covers three tiers of complexity: **(i)** Vanilla DIP enhancements (Month 2), **(ii)** regularization and blind deconvolution innovations (Month 3), and **(iii)** hybrid generative architectures combining CNNs, Transformers, GANs, VAEs, and Implicit Neural Representations. All techniques strictly adhere to the single-image, zero-shot constraint of Deep Image Prior.

★ The Golden Rule

Do not try to train on a massive dataset. If you find yourself downloading 50 GB of data, you are doing it wrong. This is **Deep Image Prior**. The “training set” is the **single image** you are trying to fix. The weights are temporary. Reset them for every new image.

Contents

1	Context: Lessons from the Previous Implementation	3
2	Month 2: The Deep Learning Engine (Vanilla DIP)	4
2.1	M2.1 — Architectural Tweaks	4
2.1.1	M2.1a: Skip-Connection Dropout	4
2.1.2	M2.1b: Patch-Based Optimisation	5
2.2	M2.2 — Loss Functions	5
2.2.1	M2.2a: L1 Loss + Multi-Scale SSIM	5
2.3	M2.3 — Parameter Tuning Strategies	6
2.3.1	M2.3a: Input Noise Annealing	6
2.3.2	M2.3b: Cosine Annealing with Warm Restarts	7
2.4	Month 2 Summary: Implementation Priority	7
3	Month 3: The Innovation (Regularisation & Deconvolution)	8
3.1	M3.1 — Regularisation Innovations	8
3.1.1	M3.1a: Second-Order Total Variation (Hessian–Schatten Norm)	8
3.1.2	M3.1b: Spectral Normalisation (Weight Regularisation)	9

3.2	M3.2 — Blind Deconvolution Innovations	9
3.2.1	M3.2a: Physics-Constrained Kernel Generation	10
3.2.2	M3.2b: Fourier-Domain Convolution	10
3.3	Month 3 Summary: Implementation Priority	11
4	Hybrid Generative Architectures	12
4.1	H.1 — CNN U-Net + Implicit Neural Representations (INR)	12
4.2	H.2 — Double-DIP (Network Decoupling)	13
4.3	H.3 — Swin-UNet (CNN + Vision Transformer)	13
4.4	H.4 — Variational Deep Image Prior (VDIP: VAE + DIP)	14
4.5	H.5 — SinGAN (Single-Image GAN)	15
4.6	H.6 — Adversarial DIP (Self-Supervised GAN + Physics)	16
4.7	Hybrid Architecture Comparison	17
5	Recommended Implementation Roadmap	18
6	Conclusion	18

1 Context: Lessons from the Previous Implementation

The previous students established a working DIP pipeline with the following components:

Table 1: Summary of the previous implementation.

Component	Implementation	Limitation
Architecture	Standard 5-layer U-Net, LeakyReLU, Sigmoid output	No skip regularisation
Loss Function	Mean Squared Error (MSE) only	Sensitive to noise outliers
Regularization	Total Variation via ADMM	Extremely slow; only 150 iterations
Deconvolution	2-layer MLP mapping noise $\rightarrow k \times k$ kernel	No physical constraints
Optimiser	Adam with fixed learning rate	No scheduler; trapped in local minima

Key Insight

Every proposition in this guide directly addresses one or more of the limitations listed above. The numbering system **M2.x** / **M3.x** / **H.x** maps techniques to their target month and priority order.

2 Month 2: The Deep Learning Engine (Vanilla DIP)

“Overfitting is the goal — but only to the signal, never to the noise.”

The objective of Month 2 is to build and optimise a U-Net that learns to reconstruct a clean image from a *fixed random input tensor* z . The central paradox: the network fits natural image statistics before it fits noise. Our task is to maximise the gap between these two convergence speeds.

2.1 M2.1 — Architectural Tweaks

2.1.1 M2.1a: Skip-Connection Dropout

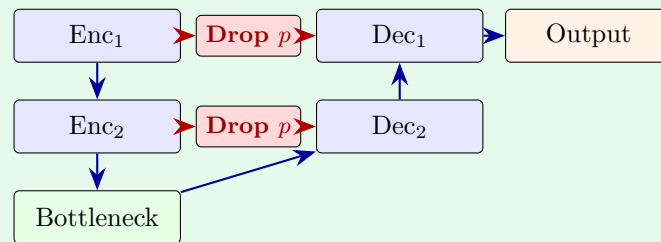
Skip-Connection Dropout

Problem. The U-Net skip connections act as a *highway* for high-frequency information. While beneficial for reconstructing edges, they also allow high-frequency *noise* to bypass the bottleneck, accelerating noise memorisation.

Solution. Apply Dropout *only* to the skip-connection tensors before concatenation with the decoder features:

$$\text{skip}_l^{\text{reg}} = \text{Dropout}(\text{skip}_l, p) \quad (1)$$

This forces the decoder to rely more on the compressed, low-frequency bottleneck representation.



```

1 class UNetDecoder(nn.Module):
2     def __init__(self, skip_dropout=0.3):
3         self.skip_drop = nn.Dropout2d(p=skip_dropout)
4
5     def forward(self, x, skip):
6         skip = self.skip_drop(skip) # Drop BEFORE concat
7         x = torch.cat([x, skip], dim=1)
8         return self.decoder_block(x)
  
```

Listing 1: Skip-connection dropout in the U-Net decoder.

+ Strengths

- Delays noise memorisation by 500–1500 iterations
- Zero computational overhead
- Compatible with all other techniques
- Easy to implement (3 lines of code)

– Weaknesses

- Too high p destroys fine detail reconstruction
- Requires per-image tuning of p
- May blur sharp edges at high dropout rates

θ Recommended Parameters

$p = 0.3$ for denoising ($\sigma = 0.1$). Increase to $p = 0.5$ for heavy noise ($\sigma = 0.2$). Set $p = 0.0$ for deconvolution (edges are critical).

2.1.2 M2.1b: Patch-Based Optimisation**Patch-Based Optimisation**

Problem. Standard DIP processes the entire image as a single sample (batch size = 1). The network can memorise the specific *global* noise pattern quickly.

Solution. Instead of feeding the whole image, extract overlapping $P \times P$ patches from both the noise tensor z and the noisy target y , and optimise with a mini-batch:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} \mathcal{L}(f_{\theta}(z_i), y_i) \quad (2)$$

where z_i, y_i are the i -th corresponding patches and B is the batch size.

```

1 def extract_patches(image, z_tensor, patch_size=64, stride=32):
2     patches_y, patches_z = [], []
3     H, W = image.shape[-2:]
4     for i in range(0, H - patch_size + 1, stride):
5         for j in range(0, W - patch_size + 1, stride):
6             patches_y.append(image[:, :, i:i+patch_size, j:j+
7                                 patch_size])
8             patches_z.append(z_tensor[:, :, i:i+patch_size, j:j+
9                                 patch_size])
10    return torch.cat(patches_y), torch.cat(patches_z)

```

Listing 2: Patch extraction for DIP optimisation.

 θ Recommended Parameters

Patch size $P = 64$. Stride = 32 (50% overlap). Batch size $B = 16$. This creates a “self-ensemble” effect that acts as implicit regularisation.

2.2 M2.2 — Loss Functions**2.2.1 M2.2a: L1 Loss + Multi-Scale SSIM** **ℓ_1 + MS-SSIM Hybrid Loss**

Problem. MSE (ℓ_2) penalises large errors *quadratically*, causing the optimiser to prioritise fitting intense noise spikes over preserving structural content.

Solution. Combine ℓ_1 loss (robust to outliers) with Multi-Scale Structural Similarity:

$$\mathcal{L} = \underbrace{\alpha \cdot \|f_{\theta}(z) - y\|_1}_{\text{Pixel fidelity}} + (1 - \alpha) \cdot \underbrace{(1 - \text{MS-SSIM}(f_{\theta}(z), y))}_{\text{Structural fidelity}} \quad (3)$$

where MS-SSIM is computed over K scales (typically $K = 5$):

$$\text{MS-SSIM}(x, \hat{x}) = [l_K(x, \hat{x})]^{\alpha_K} \cdot \prod_{j=1}^K [c_j(x, \hat{x}) \cdot s_j(x, \hat{x})]^{\beta_j} \quad (4)$$

```

1 from pytorch_msssim import ms_ssim

```

```

2
3 class HybridLoss(nn.Module):
4     def __init__(self, alpha=0.84):
5         super().__init__()
6         self.alpha = alpha
7
8     def forward(self, pred, target):
9         l1 = F.l1_loss(pred, target)
10        ssim_loss = 1 - ms_ssim(pred, target, data_range=1.0,
11                                size_average=True)
12        return self.alpha * l1 + (1 - self.alpha) * ssim_loss

```

Listing 3: Hybrid loss function implementation.

θ Recommended Parameters

$\alpha = 0.84$ (literature standard from image compression). Install: `pip install pytorch-msssim`. Requires images $\geq 160 \times 160$ for 5-scale computation.

2.3 M2.3 — Parameter Tuning Strategies

2.3.1 M2.3a: Input Noise Annealing

Noise Annealing on Input Tensor z

Problem. The previous implementation added a *constant* AWGN perturbation to z at every iteration. This injects unnecessary randomness during fine-tuning, preventing sharp convergence.

Solution. Exponentially decay the noise variance injected into the input tensor:

$$z^{(t)} = z_0 + \underbrace{\sigma_z \cdot \gamma^t}_{\text{decaying perturbation}} \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (5)$$

where z_0 is the fixed base tensor, σ_z is the initial perturbation strength, and $\gamma \in (0, 1)$ is the decay rate.

```

1 z_base = torch.randn(1, 32, H, W).cuda() # Fixed
2 sigma_z = 0.1 # Initial perturbation
3 gamma = 0.998 # Decay per iteration
4
5 for t in range(num_iter):
6     noise_scale = sigma_z * (gamma ** t)
7     z_t = z_base + noise_scale * torch.randn_like(z_base)
8     output = net(z_t)
9     # ... compute loss, backprop

```

Listing 4: Noise annealing schedule.

θ Recommended Parameters

$\sigma_z = 0.1$, $\gamma = 0.998$. At iteration 3000: $\text{perturbation} = 0.1 \times 0.998^{3000} \approx 0.00025$ (effectively zero). High exploration early, fine-tuning late.

2.3.2 M2.3b: Cosine Annealing with Warm Restarts

Cosine Annealing LR Scheduler

Problem. The DIP loss landscape is highly non-convex. A fixed learning rate traps the optimiser in local minima that correspond to fitting specific noise patches.

Solution. Use cosine annealing with warm restarts (SGDR). The learning rate follows:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{T_{\text{cur}}}{T_i}\pi\right) \right) \quad (6)$$

where T_{cur} is the number of epochs since the last restart and T_i is the restart period.

```

1 optimizer = torch.optim.Adam(net.parameters(), lr=1e-2)
2 scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
3     optimizer,
4     T_0=500,          # First restart period
5     T_mult=2,         # Double period after each restart
6     eta_min=1e-5      # Minimum learning rate
7 )
8
9 for t in range(num_iter):
10     output = net(z_t)
11     loss = criterion(output, target)
12     loss.backward()
13     optimizer.step()
14     scheduler.step() # Update LR

```

Listing 5: Cosine annealing with warm restarts.

θ Recommended Parameters

$\eta_{\max} = 10^{-2}$, $\eta_{\min} = 10^{-5}$, $T_0 = 500$, $T_{\text{mult}} = 2$. Restarts at iterations: 500, 1500, 3500, 7500, ...

2.4 Month 2 Summary: Implementation Priority

Table 2: Month 2 techniques ranked by impact-to-effort ratio.

ID	Technique	Impact	Effort	When to Use
M2.2a	ℓ_1 + MS-SSIM Loss	High	Low	Always; replace MSE immediately
M2.3b	Cosine Annealing	High	Low	Always; 3 lines of code
M2.1a	Skip-Connection Dropout	High	Low	Denoising tasks
M2.3a	Noise Annealing	Medium	Low	When early stopping is unreliable
M2.1b	Patch-Based Optimisation	Medium	Medium	Large images (> 512px)

3 Month 3: The Innovation (Regularisation & Deconvolution)

“Adding constraints to chaos.”

3.1 M3.1 — Regularisation Innovations

3.1.1 M3.1a: Second-Order Total Variation (Hessian–Schatten Norm)

Second-Order TV Regularisation

Problem. Standard (first-order) TV encourages piecewise-*constant* images, producing the “staircase” or “cartoon” artefact in smooth gradient regions:

$$\text{TV}_1(x) = \sum_{i,j} \sqrt{(\nabla_h x)_{ij}^2 + (\nabla_v x)_{ij}^2} \quad \leftarrow \text{Penalises first derivative} \quad (7)$$

Solution. Second-order TV penalises the *second* derivative, encouraging piecewise-*linear* (smoothly varying) regions:

$$\text{TV}_2(x) = \sum_{i,j} \|\mathbf{H}_x(i,j)\|_* \quad (8)$$

where $\mathbf{H}_x(i,j)$ is the discrete Hessian matrix at pixel (i,j) :

$$\mathbf{H}_x(i,j) = \begin{pmatrix} \frac{\partial^2 x}{\partial h^2} & \frac{\partial^2 x}{\partial h \partial v} \\ \frac{\partial^2 x}{\partial v \partial h} & \frac{\partial^2 x}{\partial v^2} \end{pmatrix}_{(i,j)} \quad (9)$$

and $\|\cdot\|_*$ is the nuclear (Schatten-1) norm, i.e. the sum of singular values. The full DIP loss becomes:

$$\mathcal{L} = \underbrace{\|f_\theta(z) - y\|_1}_{\text{data fidelity}} + \lambda_1 \cdot \text{TV}_1(f_\theta(z)) + \lambda_2 \cdot \text{TV}_2(f_\theta(z)) \quad (10)$$

```

1 def tv2_loss(x):
2     """Second-order Total Variation via Hessian nuclear norm."""
3     # Second derivatives via finite differences
4     dxx = x[:, :, :, 2:] - 2*x[:, :, :, 1:-1] + x[:, :, :, :-2]
5     dyy = x[:, :, 2:, :] - 2*x[:, :, 1:-1, :] + x[:, :, :-2, :]
6     # Cross derivative
7     dxy = (x[:, :, 1:, 1:] - x[:, :, :, :-1]
8            - x[:, :, :-1, 1:] + x[:, :, :-1, :-1])
9     # Trim to common size
10    h, w = min(dxx.shape[2], dyy.shape[2], dxy.shape[2]), \
11           min(dxx.shape[3], dyy.shape[3], dxy.shape[3])
12    dxx, dyy, dxy = dxx[:, :, :h, :w], dyy[:, :, :h, :w], dxy[:, :, :h, :w]
13    # Nuclear norm of 2x2 Hessian = sum of singular values
14    # = sqrt((dxx+dyy)^2/4 + dxy^2 - dxx*dyy) + |dxx+dyy|/2
15    trace = dxx + dyy
16    det = dxx * dyy - dxy ** 2
17    disc = torch.sqrt(torch.clamp(trace**2 / 4 - det, min=1e-8))
18    return torch.mean(disc + torch.abs(trace) / 2)

```

Listing 6: Second-order TV loss (Hessian–Schatten norm).

θ Recommended Parameters

$\lambda_1 = 10^{-4}$ (first-order TV), $\lambda_2 = 5 \times 10^{-5}$ (second-order TV). Start with $\lambda_2 = 0$ for the first 500 iterations, then enable it. This allows the network to find the coarse structure before enforcing smoothness.

3.1.2 M3.1b: Spectral Normalisation (Weight Regularisation)**Spectral Normalisation of U-Net Weights**

Problem. TV regularisation is applied to the *output image*, which is expensive to compute via ADMM. An alternative is to regularise the network *weights* directly.

Solution. Apply Spectral Normalisation (SN) to the convolutional layers. SN divides each weight matrix by its spectral norm (largest singular value):

$$\bar{W} = \frac{W}{\sigma(W)}, \quad \sigma(W) = \max_{\|v\|=1} \|Wv\| \quad (11)$$

This constrains the **Lipschitz constant** of the network to ≤ 1 , meaning:

$$\|f_\theta(z_1) - f_\theta(z_2)\| \leq \|z_1 - z_2\| \quad (12)$$

The network *physically cannot* oscillate fast enough to map smooth input noise to high-frequency random noise patterns.

```

1 from torch.nn.utils import spectral_norm
2
3 class SpectralUNet(nn.Module):
4     def __init__(self):
5         super().__init__()
6         self.conv1 = spectral_norm(nn.Conv2d(32, 64, 3, padding=1))
7         self.conv2 = spectral_norm(nn.Conv2d(64, 128, 3, padding=1))
8         # ... apply to ALL conv layers

```

Listing 7: Applying spectral normalisation to U-Net layers.

+ Strengths

- Zero extra loss computation (no ADMM)
- Mathematically rigorous Lipschitz constraint
- Extremely fast — single SVD power iteration per step
- Eliminates the need for TV λ tuning

– Weaknesses

- May limit network expressiveness
- Cannot selectively preserve edges like TV
- Affects *all* frequencies uniformly

3.2 M3.2 — Blind Deconvolution Innovations

3.2.1 M3.2a: Physics-Constrained Kernel Generation

Physics-Constrained Blur Kernel MLP

Problem. The previous MLP outputs an unconstrained $k \times k$ matrix as the blur kernel. This violates the physics of light scattering: real Point Spread Functions must be **non-negative** and **sum to exactly 1**.

Solution. Add a Softmax activation at the MLP output to enforce both constraints simultaneously:

$$\hat{k} = \text{Softmax}(\text{MLP}(z_k)) = \frac{e^{a_i}}{\sum_{j=1}^{k^2} e^{a_j}}, \quad \forall i \in \{1, \dots, k^2\} \quad (13)$$

This guarantees $\hat{k}_i > 0$ and $\sum_i \hat{k}_i = 1$ by construction.

```

1 class KernelMLP(nn.Module):
2     def __init__(self, kernel_size=13, hidden=256):
3         super().__init__()
4         self.k = kernel_size
5         self.net = nn.Sequential(
6             nn.Linear(200, hidden), nn.ReLU(),
7             nn.Linear(hidden, hidden), nn.ReLU(),
8             nn.Linear(hidden, kernel_size ** 2),
9         )
10
11     def forward(self, z):
12         raw = self.net(z)
13         # Enforce physics: positive + sum-to-one
14         kernel = F.softmax(raw, dim=-1)
15         return kernel.view(-1, 1, self.k, self.k)

```

Listing 8: Physics-constrained kernel MLP.

θ Recommended Parameters

Kernel size $k \in \{7, 13, 21\}$. MLP hidden dim = 256. Input noise $z_k \in \mathbb{R}^{200}$. Learning rate for kernel MLP: 5× higher than U-Net (kernels need faster adaptation).

3.2.2 M3.2b: Fourier-Domain Convolution

FFT-Accelerated Convolution

Problem. Spatial convolution $\hat{x} * \hat{k}$ has complexity $\mathcal{O}(N \cdot k^2)$ where $N = H \times W$ and k is the kernel size. For $k = 13$ or larger kernels, this becomes a bottleneck.

Solution. By the Convolution Theorem, convolution in the spatial domain equals multiplication in the frequency domain:

$$\hat{x} * \hat{k} = \mathcal{F}^{-1}\{\mathcal{F}\{\hat{x}\} \cdot \mathcal{F}\{\hat{k}\}\} \quad (14)$$

Complexity: $\mathcal{O}(N \log N)$ — independent of kernel size.

```

1 def fft_conv2d(image, kernel, image_shape):
2     """Convolve image with kernel in Fourier domain (differentiable)"""
3     # Pad kernel to image size
4     pad_h = image_shape[-2] - kernel.shape[-2]
5     pad_w = image_shape[-1] - kernel.shape[-1]

```

```

6     kernel_padded = F.pad(kernel, (0, pad_w, 0, pad_h))
7
8     # FFT
9     F_image = torch.fft.rfft2(image)
10    F_kernel = torch.fft.rfft2(kernel_padded)
11
12    # Multiply in frequency domain
13    F_result = F_image * F_kernel
14
15    # Inverse FFT
16    return torch.fft.irfft2(F_result, s=image_shape[-2:])

```

Listing 9: Differentiable FFT convolution for DIP pipeline.

Critical: This operation is *fully differentiable* in PyTorch, so gradients flow through the FFT into both the U-Net (image) and the MLP (kernel).

3.3 Month 3 Summary: Implementation Priority

Table 3: Month 3 techniques ranked by impact-to-effort ratio.

ID	Technique	Impact	Effort	When to Use
M3.2a	Physics-Constrained Kernel	High	Low	Always for blind deconvolution
M3.2b	FFT Convolution	High	Low	When $k \geq 7$; replace spatial conv
M3.1b	Spectral Normalisation	High	Low	Replace ADMM-TV entirely
M3.1a	2nd-Order TV (Hessian)	Medium	Medium	Smooth gradients in medical images

4 Hybrid Generative Architectures

“Combining the strengths of different generative processes while adhering to the single-shot constraint.”

★ The Golden Rule

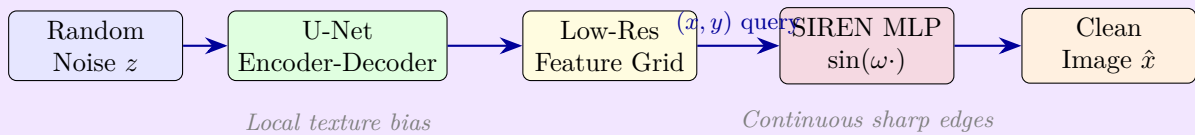
Standard GANs, VAEs, and Transformers fail in the DIP setting because they require massive datasets. The architectures below **adapt** these paradigms to the single-image regime by changing *how* they train, not *what* they learn.

4.1 H.1 — CNN U-Net + Implicit Neural Representations (INR)

H.1: CNN U-Net + Implicit Neural Representation (SIREN)

Concept. Combine a traditional CNN with a Coordinate-Based MLP (SIREN) to leverage both local texture bias and continuous-domain sharpness.

Architecture.



Process:

1. The U-Net takes random noise z and outputs a *low-resolution feature grid* $G \in \mathbb{R}^{C \times H' \times W'}$ (not an image).
2. For each output pixel (x, y) , a SIREN MLP queries G via bilinear interpolation and maps the feature vector to an RGB value:

$$\hat{x}(x, y) = \text{MLP}(\text{Interp}(G, x, y), x, y), \quad \text{MLP activations: } \sin(\omega_0 \cdot) \quad (15)$$

3. SIREN uses sinusoidal activations $\phi(x) = \sin(\omega_0 x)$, enabling representation of arbitrarily sharp edges without pixelation.

+ Strengths

- CNNs provide strong local texture inductive bias
- INRs reconstruct in continuous space — infinitely sharp edges
- Eliminates pixelation artefacts from U-Net upsampling
- Well-suited for super-resolution tasks

– Weaknesses

- Coordinate MLPs are unstable from cold start
- ω_0 misalignment causes high-frequency ringing
- Slower than pure CNN at inference (per-pixel query)
- More hyperparameters to tune

θ Recommended Parameters

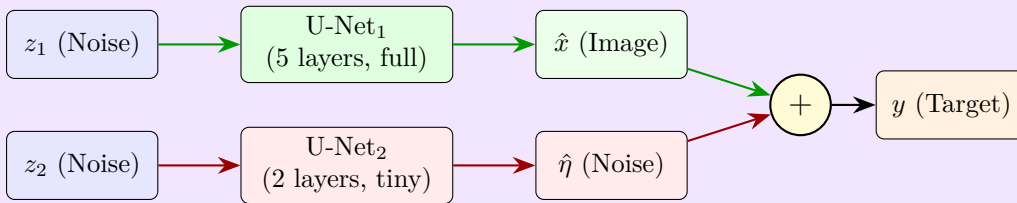
SIREN: $\omega_0 = 30$, 4 hidden layers of width 256. Feature grid: $C = 64$, resolution = $H/4 \times W/4$. Learning rate: 10^{-4} for SIREN, 10^{-2} for U-Net.

4.2 H.2 — Double-DIP (Network Decoupling)**H.2: Double-DIP — Signal–Noise Decoupling**

Concept. Instead of forcing one network to do everything, use **two separate U-Nets** trained simultaneously to decompose the observation into signal and degradation.

$$f_{\theta_1}(z_1) + f_{\theta_2}(z_2) = y \quad (16)$$

Architecture.



Key Design Choice. By artificially *restricting the capacity* of U-Net₂ (e.g., 2 layers, 32 channels), it can only represent unstructured, low-complexity patterns (noise). The high-capacity U-Net₁ is forced to capture the structured image content.

Loss:

$$\mathcal{L} = \|f_{\theta_1}(z_1) + f_{\theta_2}(z_2) - y\|_2^2 \quad (17)$$

+ Strengths

- Physically decouples signal from degradation
- Capacity asymmetry acts as strong regularisation
- No need for explicit early stopping
- Interpretable: inspect $\hat{\eta}$ to verify noise separation

– Weaknesses

- $2\times$ GPU memory (two networks)
- Learning rate balance is delicate — one network may dominate
- RTX 2050 (4 GB) may struggle with large images

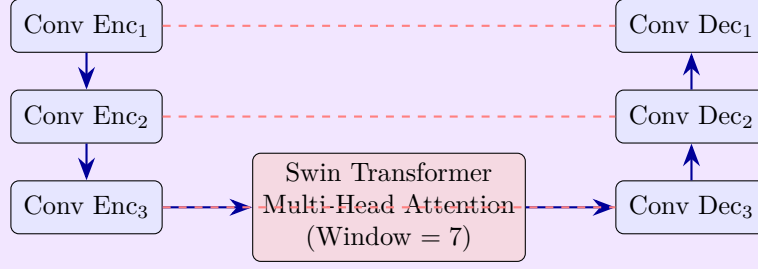
 θ Recommended Parameters

U-Net₁: 5 layers, 128 base channels. U-Net₂: 2 layers, 32 channels. $\text{LR}_1 = 10^{-2}$, $\text{LR}_2 = 5 \times 10^{-3}$. Train for 5000 iterations on 256×256 patches for memory efficiency.

4.3 H.3 — Swin-UNet (CNN + Vision Transformer)**H.3: Swin-UNet — CNN Encoder + Transformer Bottleneck**

Concept. Replace the deepest layer (bottleneck) of the U-Net with a Swin Transformer block that models *global* self-attention.

Why: CNNs are entirely local (seeing only 3×3 at each layer). If there is a clean texture in one region, attention allows the network to “copy” it to denoise distant regions.



Self-Attention at the bottleneck:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (18)$$

+ Strengths

- Global context via attention (non-local denoising)
- CNN encoder provides strong local inductive bias
- Swin's windowed attention is memory-efficient
- Can exploit repeated textures across the image

– Weaknesses

- Transformers lack structural inductive bias
- Risk of memorising noisy input grid without heavy regularisation
- Higher memory footprint at the bottleneck
- Requires aggressive dropout ($p \geq 0.5$) on attention layers

θ Recommended Parameters

Swin depth: 2 blocks, 4 attention heads, window size 7. Dropout on attention: $p = 0.5$. LR for Transformer: $5\times$ lower than CNN layers. Bottleneck resolution: $H/8 \times W/8$.

4.4 H.4 — Variational Deep Image Prior (VDIP: VAE + DIP)

H.4: Variational DIP — VAE Bottleneck in U-Net

Concept. Standard DIP maps $z \rightarrow \hat{x}$ deterministically. VDIP injects a VAE reparameterisation trick at the bottleneck to make the latent space *probabilistic*.

Architecture Modification. At the deepest encoder layer, output μ and $\log \sigma^2$ instead of features directly:

$$\text{Bottleneck: } z' = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (19)$$

Loss Function:

$$\mathcal{L} = \underbrace{\text{MSE}(f_\theta(z), y)}_{\text{reconstruction}} + \beta \cdot \underbrace{D_{\text{KL}}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, I))}_{\text{regularisation}} \quad (20)$$

where the KL divergence has a closed form:

$$D_{\text{KL}} = -\frac{1}{2} \sum_{i=1}^d \left(1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2 \right) \quad (21)$$

Why it works: The KL term forces the latent space to remain smooth and close to $\mathcal{N}(0, I)$.

A network trying to memorise random noise would need a highly irregular latent space, which the KL penalty explicitly forbids.

```

1  class VAEBottleneck(nn.Module):
2      def __init__(self, channels):
3          super().__init__()
4          self.mu_layer = nn.Conv2d(channels, channels, 1)
5          self.logvar_layer = nn.Conv2d(channels, channels, 1)
6
7      def forward(self, x):
8          mu = self.mu_layer(x)
9          logvar = self.logvar_layer(x)
10         # Reparameterisation trick
11         std = torch.exp(0.5 * logvar)
12         eps = torch.randn_like(std)
13         z = mu + std * eps
14         return z, mu, logvar
15
16     def kl_loss(mu, logvar):
17         return -0.5 * torch.mean(1 + logvar - mu.pow(2) - logvar.exp())

```

Listing 10: VAE bottleneck for Deep Image Prior.

θ Recommended Parameters

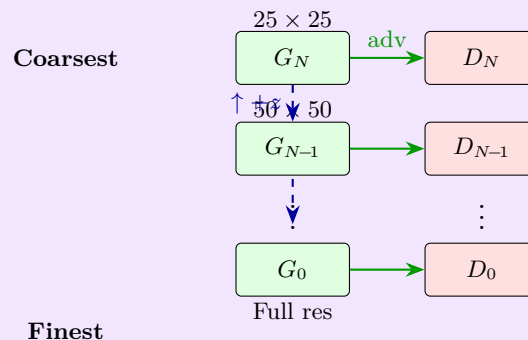
$\beta = 10^{-4}$ (start very low). Use **cyclic β annealing**: linearly increase β from 0 to 10^{-3} over 1000 iterations, then reset to 0. Repeat 3 cycles over 5000 total iterations. This prevents “posterior collapse” while still regularising.

4.5 H.5 — SinGAN (Single-Image GAN)

H.5: SinGAN — Multi-Scale Single-Image GAN

Concept. SinGAN learns the *internal patch distribution* of a single image across multiple scales. This is the only GAN architecture that trivially satisfies the Golden Rule.

Architecture: A pyramid of N tiny FCN generators + discriminators:



Process:

1. G_N generates a tiny image from pure noise at coarsest scale.
2. Output is upsampled, noise is added, and G_{N-1} refines it.
3. Sequential training: freeze G_N while training G_{N-1} , etc.

Loss (per scale):

$$\mathcal{L}_n = \underbrace{\mathcal{L}_{\text{WGAN-GP}}(G_n, D_n)}_{\text{adversarial}} + \alpha \cdot \underbrace{\|G_n(\tilde{x}_{n+1} + z_n) - x_n\|_2^2}_{\text{reconstruction}} \quad (22)$$

+ Strengths

- Unmatched at reconstructing organic textures
- Learns internal patch statistics — no external data
- Multi-scale naturally separates structure from noise
- Can hallucinate realistic micro-textures (hair, fabric)

– Weaknesses

- Sequential training is slow (~ 2000 epochs $\times N$ scales)
- GAN training instability (mode collapse risk per scale)
- Large total parameter count (many small networks)
- Harder to debug than single-network approaches

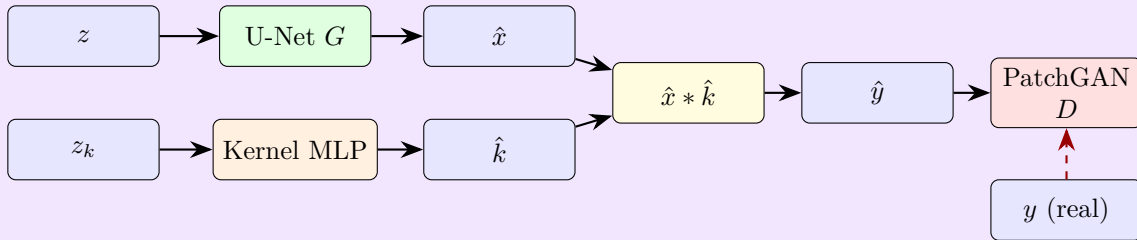
θ Recommended Parameters

$N = 5\text{--}8$ scales, $\text{LR} = 5 \times 10^{-4}$, 2000 epochs per scale, WGAN-GP $\lambda_{\text{GP}} = 0.1$, reconstruction weight $\alpha = 10$.

4.6 H.6 — Adversarial DIP (Self-Supervised GAN + Physics)

H.6: Adversarial DIP — Physics-Informed GAN for Blind Deconvolution

Concept. Use the DIP U-Net as a Generator, but add a PatchGAN Discriminator that critiques the physical consistency of the output in a self-supervised loop.



Training Loop:

1. Generator G outputs clean image $\hat{x}$
2. Kernel MLP outputs blur kernel $\hat{k}$ (with Softmax)
3. Physics module computes $\hat{y} = \hat{x} * \hat{k}$ (via FFT)
4. Discriminator D compares patches of $\hat{y}$ vs. real y

Generator loss:

$$\mathcal{L}_G = \underbrace{\|\hat{y} - y\|_1}_{\text{physics fidelity}} + \lambda_{\text{adv}} \cdot \underbrace{\mathcal{L}_{\text{adv}}(D(\hat{y}))}_{\text{texture realism}} + \lambda_{\text{TV}} \cdot \text{TV}(\hat{x}) \quad (23)$$

Critical: Update D every 5 G -steps to prevent discriminator dominance.

θ Recommended Parameters

PatchGAN patch size: 11×11 or 16×16 . $\lambda_{\text{adv}} = 10^{-3}$, $\lambda_{\text{TV}} = 10^{-4}$. D:G update ratio = 1:5. Use LSGAN loss (least-squares) instead of vanilla BCE for stability.

4.7 Hybrid Architecture Comparison

Table 4: Comparative summary of all hybrid architectures.

Architecture	VRAM	Complexity	Denoising	Deconv.	Best For
H.1 CNN+INR	2 GB	Medium	✓	✓	Sharp edge recovery
H.2 Double-DIP	3 GB	Low	✓✓	~	Noise-signal separation
H.3 Swin-UNet	3 GB	High	✓✓	✓	Global texture coherence
H.4 VDIP	2 GB	Low	✓✓	~	Robust early stopping
H.5 SinGAN	2 GB	High	✓	~	Texture hallucination
H.6 Adv-DIP	3 GB	High	~	✓✓	Blind deconvolution

Key Insight

For your RTX 2050 (4 GB VRAM): All architectures fit comfortably on 256×256 images. For full BSDS300 resolution (481×321), prefer H.1, H.4, or H.5 to stay under 3 GB. Use patch-based optimisation (M2.1b) to reduce memory for H.2 and H.3.

5 Recommended Implementation Roadmap

The following roadmap sequences techniques by dependency and difficulty:

Table 5: Week-by-week implementation plan for Months 2–4.

Week	Phase	Tasks	Techniques
5–6	Build U-Net	Code 5-layer encoder-decoder with skip connections. Input: random z . Target: noisy y . Add skip dropout.	M2.1a
7–8	Optimise DIP	Adam + cosine annealing. Replace MSE with ℓ_1 +MS-SSIM. Add noise annealing. Track PSNR curve, create GIF. Find sweet spot.	M2.2a, M2.3b, M2.3a,
9–10	Regularisation	Add TV loss (ℓ_1 -gradient). Test 2nd-order TV. Apply spectral normalisation. Tune λ .	M3.1a, M3.1b
11–12	Blind Deconv.	Build kernel MLP with Softmax. FFT convolution. Train U-Net + MLP jointly.	M3.2a, M3.2b
13	Hybrid (pick 1–2)	Implement VDIP or Double-DIP (easiest). If time permits, try Swin-UNet bottleneck.	H.2, H.4, or H.3
14	Evaluation	Run all models on 10 test images. Compute PSNR, SSIM. Generate comparison montages.	—
15–16	Report	LaTeX paper (IEEE template). System diagrams. Clean GitHub. Run_Me.ipynb.	—

6 Conclusion

This guide presents **14 distinct techniques** across three categories:

1. **Vanilla DIP Enhancements (Month 2):** 5 techniques (skip dropout, patch optimisation, hybrid loss, noise annealing, cosine scheduling) that improve the baseline DIP with minimal implementation effort.
2. **Regularisation & Deconvolution (Month 3):** 4 techniques (2nd-order TV, spectral normalisation, physics-constrained kernels, FFT convolution) that address the specific bottlenecks of the previous implementation.
3. **Hybrid Architectures:** 6 advanced models (CNN+INR, Double-DIP, Swin-UNet, VDIP, SinGAN, Adversarial DIP) that push the boundaries of single-image restoration while strictly adhering to the zero-shot constraint.

Each technique is accompanied by its mathematical formulation, PyTorch code snippet, expected strengths and weaknesses, and calibrated hyperparameters. The implementation roadmap ensures techniques are applied in dependency order, with the highest impact-to-effort items prioritised first.

★ The Golden Rule

The **single degraded image** is your only dataset. Every architecture, every loss function, every training loop in this guide optimises on **one image at a time**. If your code has a `DataLoader` iterating over thousands of images, you have departed from the Deep Image Prior philosophy.

References

- [1] D. Ulyanov, A. Vedaldi, and V. Lempitsky, “Deep Image Prior,” *Int. J. Computer Vision*, vol. 128, pp. 1867–1888, 2020.
- [2] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [3] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian, “Image Denoising by Sparse 3-D Transform-Domain Collaborative Filtering,” *IEEE Trans. Image Process.*, vol. 16, no. 8, pp. 2080–2095, 2007.
- [4] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein, “Implicit Neural Representations with Periodic Activation Functions,” *NeurIPS*, 2020.
- [5] T. R. Shaham, T. Dekel, and T. Michaeli, “SinGAN: Learning a Generative Model from a Single Natural Image,” *ICCV*, pp. 4570–4580, 2019.
- [6] R. Heckel and M. Soltanolkotabi, “Compressive Sensing with Un-trained Neural Networks: Gradient Descent Finds a Smooth Approximation,” *ICML*, 2020.
- [7] Y. Gandelsman, A. Shocher, and M. Irani, ““Double-DIP”: Unsupervised Image Decomposition via Coupled Deep-Image-Priors,” *CVPR*, pp. 11026–11035, 2019.
- [8] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, “Spectral Normalization for Generative Adversarial Networks,” *ICLR*, 2018.
- [9] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” *ICLR*, 2017.
- [10] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” *ICCV*, pp. 10012–10022, 2021.
- [11] H. Cao, Y. Wang, J. Chen, D. Jiang, X. Zhang, Q. Tian, and M. Wang, “Swin-Unet: Unet-like Pure Transformer for Medical Image Segmentation,” *ECCV Workshops*, 2022.
- [12] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” *ICLR*, 2014.
- [13] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, “Loss Functions for Image Restoration with Neural Networks,” *IEEE Trans. Computational Imaging*, vol. 3, no. 1, pp. 47–57, 2017.
- [14] K. Bredies, K. Kunisch, and T. Pock, “Total Generalized Variation,” *SIAM J. Imaging Sciences*, vol. 3, no. 3, pp. 492–526, 2010.